

DevOps Blueprint

1. CI/CD – End-to-End Flow

Concept: Code → Build → Test → Deploy → Monitor

Example:

- Developer pushes code → Jenkins triggers build
 - Docker image built and pushed to ECR
 - Helm deploys new version to Kubernetes
 - Prometheus and Grafana monitor metrics → alerts sent on Slack
- Understand:** Each stage, failure point, and rollback trigger
-

2. Blue-Green vs Canary Deployments

- **Blue-Green:** Two environments for instant switch. Best for risk-free rollbacks.
 - **Canary:** Gradual rollout to small traffic percentage, monitor, then full rollout if healthy. Use Canary for user-impact testing, Blue-Green for fast rollback.
-

3. Kubernetes Pod Scheduling

Process:

1. API Server receives request
2. Scheduler checks resource requests and node capacity
3. Scheduler binds Pod → Kubelet pulls image → starts container

Tip: Mention `kubectl describe pod` and events when debugging pending Pods.

4. Debugging Pods (CrashLoopBackOff / Pending)

- Check logs: `kubectl logs <pod> --previous`
 - Check describe: `kubectl describe pod <pod>`
 - Pending → node constraints, taints, PVC
- Use `kubectl get events` for reason codes.
-

5. Terraform Failures / State Lock

- Partial infrastructure creation if apply fails midway.
 - Use `terraform state list` and `terraform refresh`.
 - State lock prevents parallel runs overwriting infrastructure. Backend (S3 + DynamoDB) recommended for remote state locking.
-

6. Handling Secrets Securely

- **Kubernetes:** Sealed Secrets, HashiCorp Vault, KMS.
 - **Terraform:** Use `vault provider`, never plain text variables.
Avoid committing `.tfstate` or secrets to Git.
-

7. Logging and Monitoring

- Centralized logging: Fluentd, Loki
 - Metrics: Prometheus and Grafana
 - Traces: Jaeger
- Observability = Logs + Metrics + Traces**
-

8. AWS Cost Optimization

- Rightsize EC2s using CloudWatch metrics
 - Use Spot Instances and Savings Plans
 - Apply lifecycle policies for S3
- Enable cost explorer alerts.
-

9. CI/CD Security

- Sign artifacts (cosign, SLSA)
 - Use least-privilege IAM roles
 - Run SAST and DAST (Trivy, ZAP)
- Pipeline security is part of supply chain security.
-

10. GitOps

- Declarative and pull-based (ArgoCD, Flux)
 - Infrastructure always matches Git.
- GitOps = Git to cluster sync (not CI pushing to cluster).
-

11. Recovery Strategy

- Helm rollback or `kubectl rollout undo`
 - Automated rollback using health probes or Jenkins pipeline logic.
-

12. Service Mesh vs Ingress

- **Ingress:** Handles external to internal routing.
 - **Service Mesh:** Manages service-to-service traffic (Layer 7, mTLS, telemetry).
-

13. Container Networking

- CNI (Flannel, Calico) creates a virtual network.
 - Pod IPs communicate via overlay networks.
-

14. ReplicaSet vs Deployment vs StatefulSet

- **ReplicaSet:** Ensures the desired number of Pods.
 - **Deployment:** Manages rollout and rollback.
 - **StatefulSet:** Maintains identity and storage per Pod.
-

15. SQS vs Kafka

- **SQS:** Managed, simple queue, at-least-once delivery.
 - **Kafka:** High throughput, streaming, topic-based.
Use SQS for decoupling, Kafka for event streams.
-

16. Scaling

- Horizontal: Add more Pods or instances.
 - Vertical: Increase CPU/RAM per node.
Mention HPA and Cluster Autoscaler.
-

17. Observability Setup

- Logs: Loki
 - Metrics: Prometheus
 - Traces: Jaeger
Visualize everything in Grafana.
-

18. Mutable vs Immutable Infrastructure

- **Mutable:** Manual changes (pets).
 - **Immutable:** Rebuild from images (cattle).
Use image-based deployments for safer, consistent environments.
-

19. Troubleshooting at 2AM

1. Check metrics (CPU, memory, pods).
 2. Review logs, events, and health probes.
 3. Rollback if degraded.
Stay calm, restore service, then analyze root cause.
-

Summary:

Interviewers want reasoning, not recall.

Don't just say *what* → explain *why* and *how it behaves in production*.